Sesión 9

Transformers y Atención Multi-Head

Carlos César Sánchez Coronel

NLP

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

El fin de las recurrencias

- Hasta 2017, las RNN/LSTM dominaban el NLP.
- Limitaciones:
  - **Procesamiento secuencial**: no paralelizable, lentas en GPU.
  - **Dependencias largas**: a pesar de LSTM, aún sufren gradiente desvaneciente.
  - **Memoria limitada**: estado oculto comprime toda la información.
- **Solución**: Transformers (Vaswani et al., 2017) – "Attention is All You Need".

Ventajas de los Transformers

- **Paralelización total**: todas las posiciones se procesan simultáneamente.
- **Conexiones directas**: cada palabra atiende a cualquier otra en una sola capa.
- **Escalabilidad**: modelos con miles de millones de parámetros (GPT, BERT).

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Visión general

Encoder

- N bloques idénticos (típicamente 6).
- Cada bloque: Multi-Head Self-Attention + Feed-Forward Network.
- Conexiones residuales y layer norm.

Decoder

- N bloques idénticos.
- Añade una capa de Cross-Attention.
- Self-Attention enmascarada (para autoregresividad).

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Intuición de self-attention

- Cada palabra calcula la relevancia de las demás para actualizar su representación.
- Analogía: reunión donde cada persona (token) pregunta a las demás qué tan relevantes son.

Formulación matemática

Para una secuencia de T palabras representada por $\mathbf{X} \in \mathbb{R}^{T \times d}$:

1. Proyectar en Queries, Keys, Values:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

2. Calcular puntuaciones: $\mathbf{S} = \mathbf{Q}\mathbf{K}^T \in \mathbb{R}^{T \times T}$
3. Escalar: $\mathbf{S}_{\text{scaled}} = \frac{\mathbf{S}}{\sqrt{d_k}}$
4. Normalizar con softmax: $\mathbf{A} = \text{softmax}(\mathbf{S}_{\text{scaled}})$
5. Salida: $\mathbf{Z} = \mathbf{A}\mathbf{V}$

Escalado del producto punto

- $\frac{1}{\sqrt{d_k}}$ evita que las puntuaciones sean demasiado grandes.
- Si los productos punto son grandes, el softmax se satura y los gradientes se vuelven muy pequeños.
- Mejora la estabilidad numérica.

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Motivación

- Una sola cabeza puede promediar demasiado.
- Diferentes cabezas pueden aprender diferentes tipos de relaciones:
  - Sintácticas (sujeto-verbo).
  - Semánticas (sinónimos).
  - De distancia.

Formulación

1. Para cada cabeza i: $\mathbf{Q}_i = \mathbf{X}\mathbf{W}_i^Q$, $\mathbf{K}_i = \mathbf{X}\mathbf{W}_i^K$, $\mathbf{V}_i = \mathbf{X}\mathbf{W}_i^V$.
2. Calcular atención por cabeza: $\mathbf{Z}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$.
3. Concatenar: $\mathbf{Z} = \text{Concat}(\mathbf{Z}_1, \ldots, \mathbf{Z}_h)$.
4. Proyectar: $\mathbf{Z}_{\text{final}} = \mathbf{Z}\mathbf{W}^O$.

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Necesidad de positional encoding

- Self-attention es invariante a la posición: permutar las palabras produce las mismas representaciones.
- Para inyectar información de orden, se suman positional encodings a los embeddings de entrada.

Fórmula de positional encoding

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- Funciones periódicas que permiten extrapolar a longitudes no vistas.
- No añaden parámetros entrenables.

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

FFN en transformers

Arquitectura

$$\text{FFN}(x) = \text{máx}(0, x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

- Capa lineal con expansión (típicamente $4 \times d_{\text{model}}$).
- Activación ReLU (o GELU).
- Misma FFN para todas las posiciones, pero con parámetros compartidos.

- Aporta no linealidad y capacidad de transformación por token.

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Conexiones residuales

Fórmula

$$x' = x + \text{Sublayer}(x)$$

- Facilitan el flujo del gradiente en redes profundas.
- Permiten apilar muchas capas (ej. BERT Large: 24 capas).

Layer Normalization

Fórmula

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \odot \gamma + \beta$$

- Normaliza las activaciones a lo largo de la dimensión de características.
- Independiente del batch size, funciona bien con secuencias de longitud variable.

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Encoder block

1. Multi-Head Self-Attention con conexión residual + layer norm.
2. Feed-Forward Network con conexión residual + layer norm.

Decoder block

1. **Masked Multi-Head Self-Attention**: evita que el decoder vea tokens futuros (máscara triangular).
2. **Cross-Attention**: queries del decoder, keys/values del encoder.
3. Feed-Forward Network.

Cada subcapa con residual y layer norm.

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Tabla comparativa

Característica	**Transformer**	**RNN/LSTM**
Paralelización	Total (todas las posiciones)	Secuencial
Dependencias largas	Conexiones directas	Gradiente desvaneciente
Memoria	Acceso directo a todas las posiciones	Estado oculto comprimido
Entrenamiento	Rápido en GPU	Lento
Escalabilidad	Miles de millones de parámetros	Difícil de escalar

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

BERT (Bidirectional Encoder Representations from Transformers)

- Usa solo el encoder (bidireccional).
- Entrenado con Masked Language Model (MLM) y Next Sentence Prediction (NSP).
- Ideal para tareas de comprensión (clasificación, NER, QA).

GPT (Generative Pre-trained Transformer)

- Usa solo el decoder (autoregresivo).
- Entrenado con modelado de lenguaje (predecir la siguiente palabra).
- Ideal para generación de texto (ChatGPT).

T5 (Text-to-Text Transfer Transformer)

- Arquitectura encoder-decoder.
- Formula todas las tareas como texto a texto.
- Ventaja: mismo pre-entrenamiento y fine-tuning para múltiples tareas.

Vision Transformer (ViT)

- Aplica transformers a imágenes dividiéndolas en parches.
- Cada parche se proyecta linealmente y se añade positional encoding.
- Captura relaciones globales desde las primeras capas.

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Self-attention en Python

```python
import numpy as np

def softmax(x, axis=-1):
    e_x = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e_x / np.sum(e_x, axis=axis, keepdims=True)

def self_attention(X, W_q, W_k, W_v):
    # X: (T, d)
    Q = X @ W_q  # (T, d_k)
    K = X @ W_k  # (T, d_k)
    V = X @ W_v  # (T, d_k)
    scores = Q @ K.T  # (T, T)
    scaled = scores / np.sqrt(Q.shape[-1])
    weights = softmax(scaled)  # (T, T)
    output = weights @ V  # (T, d_k)
    return output, weights

# Ejemplo
T, d, d_k = 4, 8, 4
X = np.random.rand(T, d)
W_q = np.random.rand(d, d_k)
W_k = np.random.rand(d, d_k)
W_v = np.random.rand(d, d_k)
out, attn = self_attention(X, W_q, W_k, W_v)
print("Matriz de atenci n:\n", attn)
```

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Preguntas estrella

- ¿Cómo funciona el transformer y qué problema resuelve?
- ¿Qué es positional encoding y por qué es necesario?
- Diferencia entre self-attention y cross-attention.
- Explica multi-head attention.
- ¿Por qué se escala el producto punto en atención?

Preguntas avanzadas

- ¿Qué es la máscara en el decoder?
- ¿Cómo se calcula la complejidad computacional de la atención? $O(T^2 \cdot d)$
- ¿Qué ventajas tiene la atención relativa?
- ¿Cómo manejan contextos largos modelos como GPT-4? (atención dispersa, attention sinks)
- ¿Qué técnicas de optimización se usan para desplegar transformers en móviles? (pruning, quantization, distillation)

Contenido

1. Introducción
2. Arquitectura General del Transformer
3. Self-Attention
4. Multi-Head Attention
5. Positional Encoding
6. Feed-Forward Network (FFN)
7. Conexiones Residuales y Layer Normalization
8. Bloques Encoder y Decoder
9. Comparación Transformer vs RNN/LSTM
10. Modelos Derivados
11. Implementación Conceptual
12. Preparación para Entrevistas
13. Resumen

Lo que hemos aprendido

Conceptos clave

- **Transformer**: arquitectura basada únicamente en atención, sin recurrencia.
- **Self-attention**: cada palabra atiende a todas las demás.
- **Multi-head attention**: varias cabezas capturan diferentes relaciones.
- **Positional encoding**: inyecta información de orden.
- **Encoder-decoder**: bloques con atención, FFN, residual y layer norm.
- **Ventajas**: paralelización, dependencias largas, escalabilidad.
- **Modelos derivados**: BERT (encoder), GPT (decoder), T5 (encoder-decoder), ViT (imágenes).
- **Desafíos**: complejidad cuadrática, inferencia en dispositivos pequeños.

¡Gracias!